

GR5070 GIS Cheat Sheet

Table of Contents

Coordinate Reference Systems (CRS)	3
Commonly Used CRS	3
Geographic Units in the US	4
GEOID Structure	4
Geocoding	5
Get an API Key	5
Access Your API Key	6
Geocoding with QGIS	7
Geocoding with R	7
Spatial Regression (tbc.)	9
Key Concepts	9
Spatial Weights Matrix	9
Queen's Contiguity	9
Spatial Autocorrelation	9
Moran's I / Global Moran's I	9
Local Moran's I	10
LISA	10
LISA Map	10
Spatial Regression Models	10
Spatial Lag Model (SLM)	10
Spatial Error Model (SEM)	10
Spatial Durbin Model (SDM)	11
Working with GeoDa	11
Working with R	11
GIS Research with Geoda	12
Geoda Cheatsheet	12
GIS Research with R	13
Import Data	13
Getting Census Data using `tidycensus`	13
Get an API key	13
R codes	13
Set CRS	14
Fix Geometries	15
Perform A Spatial Join	15
Aggregate Points by Polygons	15
Map Making with R	16

Make Maps using `tmap`	16
Make Interactive Maps using `leaflet`	17
GIS Research with QGIS	18
Import Data	18
Set CRS	18
Processing with Spatial Layers	19
Spatial Relationships in QGIS	19
Fix Geometries	21
Generate Boundary Lines	21
Perform Spatial Intersection	21
Select Spatial Features by Location	21
Create Buffers	21
Count Points in Polygon	21
Set Point Size Based on Variables	22
Join Attributes by Location	22
Processing with Tables	22
SQL in QGIS	22
Add/Change Variables.	23
Select Spatial Features Using an Expression	23
Join Tables by ID	24
Data Resources	25
ACS Data	25
US Shapefiles	25
Appendix 1: State FIPS Code	26
Appendix 2: SQL Cheatsheet	27

Coordinate Reference Systems (CRS)

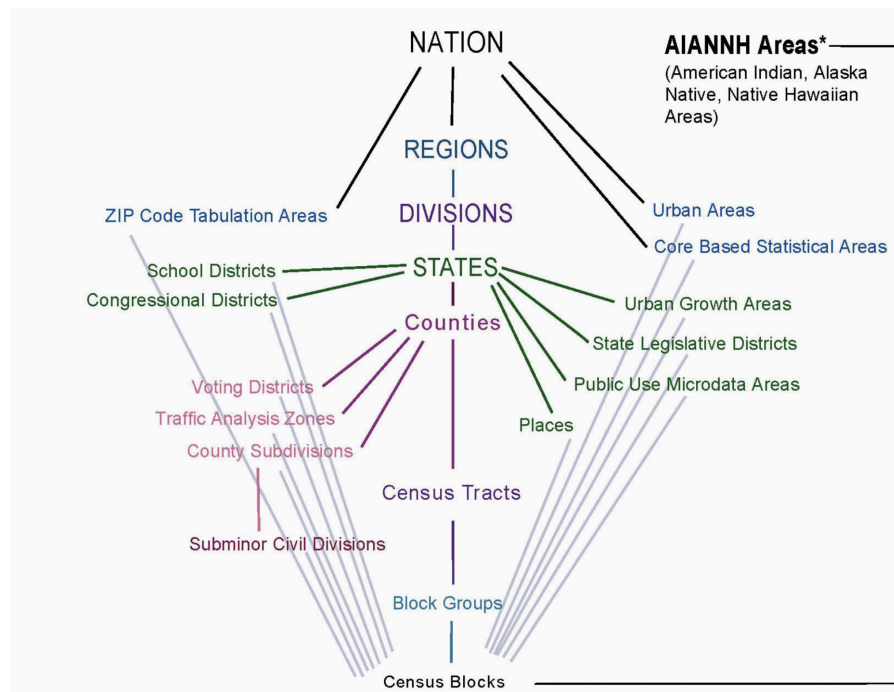
A coordinate reference system (CRS) refers to the way in which spatial data that represent the earth's surface (which is round / 3D) are flattened so that you can “Draw” them on a 2D surface.



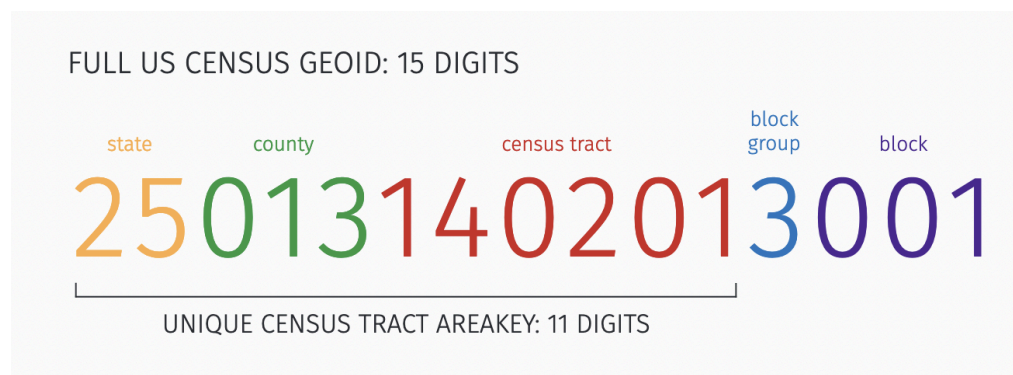
Commonly Used CRS

- Global CRS (Geographic Coordinate Systems - GCS): **WGS 84 (EPSG: 4326)**
- Common CRS for the United States: **NAD 83 (EPSG: 4269)**
- Web Mapping CRS: **WGS 84 (EPSG: 4326) / Pseudo-Mercator (EPSG: 3857)** Used by Google Maps, OpenStreetMap, Bing Maps

Geographic Units in the US



GEOID Structure

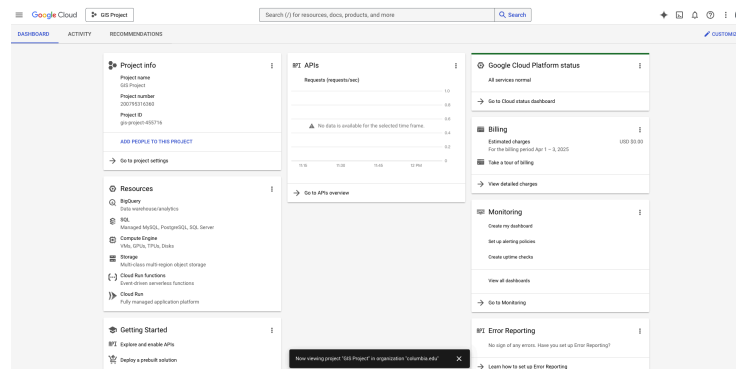


The State FIPS Code Table can be viewed here: [Appendix 1: State FIPS Code](#)

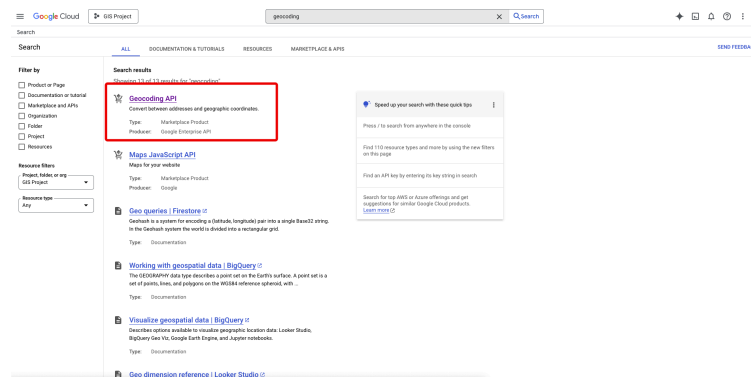
Geocoding

Get an API Key

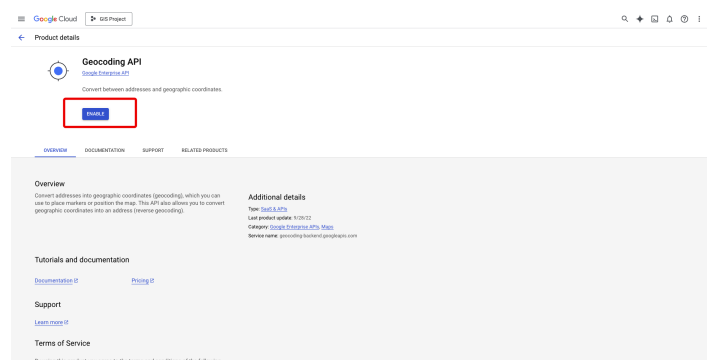
1. Open this link: <https://cloud.google.com/>
 - Sign up for an account using your .edu email address if you do not have one.
2. Go to Dashboard, and create a new project.



3. Search for “geocoding” in the search bar at the top of the screen, and click on the first result.



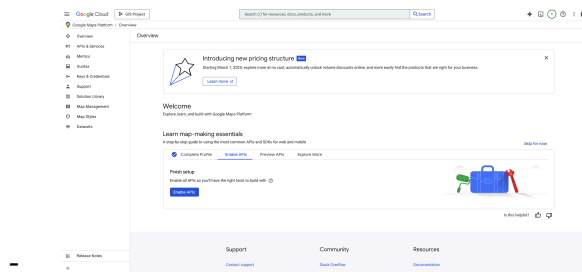
4. Click on the “enable” button.



5. After enabling the API, your API key will be automatically generated and displayed in a pop-up window (make sure to copy and save it securely). You will then be redirected to the Google Maps Platform.

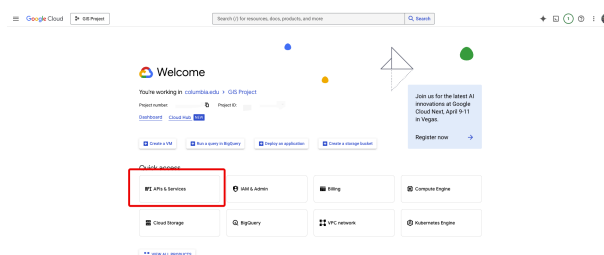
- Another pop-up titled “Protect your API key” will appear after viewing your API key. It’s fine to click on maybe later, or you can customize your settings to protect and restrict your key.

6. Then, you are all set!

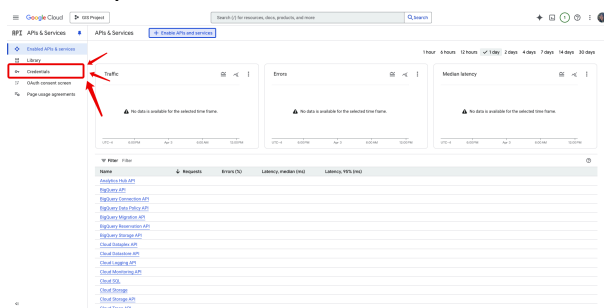


Access Your API Key

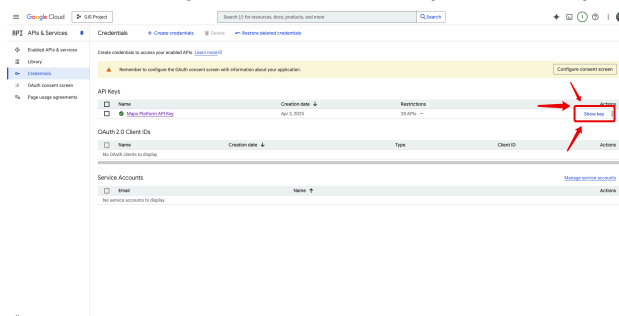
1. Log in to Google Cloud, and make sure you are in the correct account and project.
2. Click on the “APIs & Services”



3. On the left-side panel, click on “credentials”

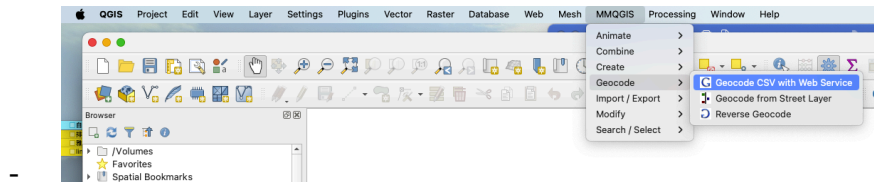


4. Click on the “show key” to view and use your API key.



Geocoding with QGIS

1. Open QGIS and navigate to the MMQGIS plugin on the top menu bar. Go to **Geocode** and select the first option from the dropdown menu.



2. Choose the appropriate .csv file that contains your address column.
 - Columns for city, state, and country are optional but helpful to improve geocoding accuracy.
3. Paste your API key into the designated field.
4. Click the Apply button to start geocoding.
5. Wait a couple of minutes for the process to complete.

Geocoding with R

If you are comfortable working with R, you can use the `tidygeocoder` package, which gives you relatively straightforward feedback in the RStudio console.

The `geocode()` function adds 2 new columns (lat and long) to the original dataset.

1. `google` Method
 - `method = "google"` allows you to request google api calls in R.

```
install.packages("tidygeocoder") # if not installed
library(tidygeocoder)
library(dplyr)
```

```
Sys.setenv(GOOGLEGEOCODE_API_KEY = "YOUR_API_KEY_HERE")
address_df <- address_df %>%
  geocode(address = address_variable_name, method = "google")
```

2. `osm` Method

- Open Street Map is API-free but slower than Google Geocoding. Feel free to use that if you have a limited number of addresses to geocode.

```
install.packages("tidygeocoder") # if not installed
library(tidygeocoder)
library(dplyr)

address_df <- address_df %>%
  geocode(address = address_variable_name, method = "osm")
```

Details on `tidygeocoder` can be viewed here: [Getting Started](#)

Spatial Regression (tbc.)

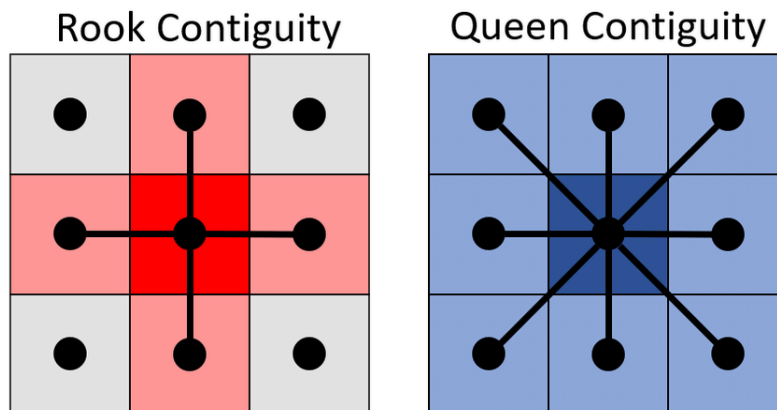
Key Concepts

Spatial Weights Matrix

A **spatial weights matrix** is a table that defines the spatial relationships or influence between geographic units, often based on adjacency or distance.

Queen's Contiguity

Queen's Contiguity is a rule for defining neighbors when creating a spatial weights matrix. It defines neighbors as spatial units that share either a border or a corner point, like the queen's move in chess.



Spatial Autocorrelation

Spatial autocorrelation refers to the degree to which a set of spatial data points resemble or differ from each other based on their location.

- Positive spatial autocorrelation: Similar values cluster together.
- Negative spatial autocorrelation: Dissimilar values are near each other.
- No Autocorrelation: Random distribution.

Moran's I / Global Moran's I

Moran's I statistic measures spatial autocorrelation, indicating whether values are randomly distributed or show clustering. **Global Moran's I** is another expression of Moran's I.

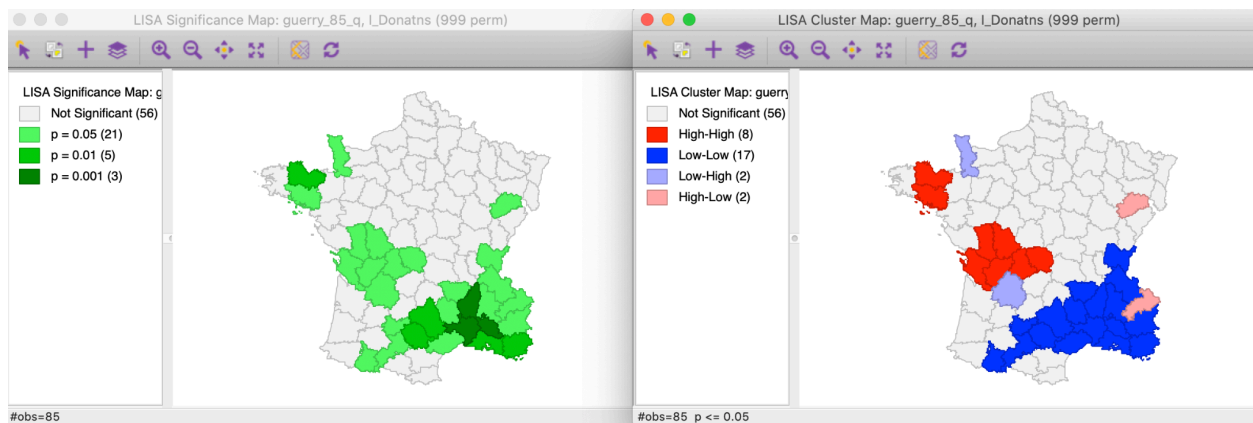
Local Moran's I

Local Moran's I statistic measures LISA.

LISA

LISA refers to local indicators of spatial association.

LISA Map



- The LISA significance map shows the locations with a significant local statistic. (shown on the left)
- The LISA cluster map shows the significant locations with an indication of the type of spatial association, based on the location of the value and its spatial lag in the Moran scatter plot. (shown on the right)

Spatial Regression Models

Spatial regression models account for spatial dependence by incorporating spatial structures.

Spatial Lag Model (SLM)

The **spatial lag model** incorporates neighboring dependent variables as **predictors**.

- $Y = \rho WY + X\beta + \epsilon$, where
 - ρWY is the spatially lagged Y value of our neighbors

Spatial Error Model (SEM)

The **spatial error model** accounts for spatial dependence in **errors**.

- $Y = X\beta + u, u = \lambda Wu + \epsilon$, where
 - $\lambda Wu + \epsilon$ is the function of our unexplained error (residuals) and our neighbor's residual values

Spatial Durbin Model (SDM)

The **spatial durbin model** captures both local and spillover effects. It is useful when policy interventions or behaviors spill across borders. (eg. How your neighbours' characteristics – WX influence your outcome.)

- $Y = \rho WY + X\beta + WX\theta + \epsilon$, where
 - WY is the spatial lag of the dependent variable (captures influence from neighbors' outcomes), and
 - WX refers to the spatial lag of the independent variables (captures influence from neighbors' characteristics)

Working with GeoDa

Working with R

GIS Research with Geoda

Geoda Cheatsheet

can be found at: <https://geodacenter.github.io/cheatsheet.html>

GIS Research with R

Import Data

- Read shapefiles (polygons, lines, etc.)

```
library(sf)          # For handling spatial data
tracts <- st_read("file_path/file.shp")
```

- Read point layers

```
# option 1
```

```
library(data.table)
points <- fread("point_data.csv", stringsAsFactors = F, data.table = F)
```

```
# option 2
```

```
library(readr)
points <- read_csv("point_data.csv")
```

```
# Convert arrest data into an sf object
```

```
library(sf)
points_sf <- st_as_sf(points, coords = c("longitude_var_name",
"latitude_var_name"), crs = 4326)
```

Getting Census Data using `tidycensus`

Get an API key

https://api.census.gov/data/key_signup.html

You may request one using the link above. (OR alternatively, request an API key from the course faculty.)

Usage Example

```
if (!require("tidycensus")) install.packages("tidycensus")
library(tidycensus)
census_api_key("YOUR_API_KEY_HERE", install = TRUE, overwrite = TRUE)
acs_vars <- c(
  total_pop = "B01003_001",
  black = "B02001_003",
```

```

hispanic = "B03003_003",
white_non_hispanic = "B03002_003",
median_income = "B19013_001",
poverty_1 = "C17002_001",
poverty_2 = "C17002_002",
poverty_3 = "C17002_003",
public_assist = "B19057_002",
female_headed = "B11003_016",
under18_poverty = "C17002_002",
foreign_born = "B05002_013",
male_18_19 = "B01001_007",
male_20 = "B01001_008",
male_21 = "B01001_009",
male_22_24 = "B01001_010",
male_25_29 = "B01001_011",
male_30_34 = "B01001_012",
female_18_19 = "B01001_031",
female_20 = "B01001_032",
female_21 = "B01001_033",
female_22_24 = "B01001_034",
female_25_29 = "B01001_035",
female_30_34 = "B01001_036"
)
nyc_acs_2010 <- get_acs(
  geography = "tract",
  variables = acs_vars,
  year = 2010,      #choose the year you are interested in
  survey = "acs5",  # choose from either "acs1" or "acs5"
  state = "NY",
  county = c("005", "047", "061", "081", "085"), # Bronx, Brooklyn,
  Manhattan, Queens, Staten Island
  geometry = FALSE,
  output = "wide"
)

```

Set CRS

```

library(sf)
# Reproject to WGS84 (lat-long)

```

```
polygons_sf <- st_transform(polygons_sf, crs = 4326)
```

Fix Geometries

```
library(sf)

invalid_geometries <- st_is_valid(polygons_sf)

if (any(!invalid_geometries)) { # Attempt to fix invalid geometries
  polygons_sf <- st_make_valid(polygons_sf)
}
```

Perform A Spatial Join

The default join is a left join, based on spatial intersection.

```
points_sf <- st_join(points_sf, polygons_sf)
```

- The above command joins attributes from *polygons_sf* to *points_sf* based on their spatial relationship.
- For example, if *points_sf* contains school locations and *polygons_sf* contains census tract GEOIDs, this operation adds the neighborhood information to each school point based on which polygon it falls within.

Aggregate Points by Polygons

There are 3 steps to count points or compute summary statistics within polygons using R.

1. Perform a spatial join to create a long table (i.e., each point is associated with its corresponding polygon).
2. Summarize the data by aggregating counts (or other statistics such as means, medians, etc.) within each geographic unit of interest.
3. Join the summarized counts back to the original polygon layer. (No spatial join is needed here. Since this step does not involve any spatial information, a simple left join on the attribute tables will be good.)

Sample Codes:

```
library(dplyr)
```

```
library(sf)

# Step 1: Spatial join - attach points to polygons
points_by_tract <- st_join(points_sf, polygons_sf, join = st_within)

# Step 2: Count the number of points per polygon
point_counts <- points_by_tract %>%
  st_drop_geometry() %>% # drop geometry, this step makes execution faster
  group_by(GEOID) %>%
  summarise(arrests = n())
# Step 3 on the next page.
# Step 3: Join counts back to polygon attribute table
polygons_sf <- left_join(polygons_sf, point_counts, by = "GEOID")
```

Map Making with R

A detailed tutorial on map-making with R can be found at:

https://bookdown.org/nicohahn/making_maps_with_r5/docs/introduction.html

Make Maps using `tmap`

```
library(tmap)

# Draws polygon boundaries with default styling.
tm_shape(polygons_sf) +
  tm_borders()

# Fills polygons based on the population variable (choropleth map).
tm_shape(polygons_sf) +
  tm_fill("population") +
  tm_borders()

# Adds a layer of points on top of polygons.
tm_shape(polygons_sf) +
  tm_borders() +
```



```
tm_shape(points_sf) +  
  tm_dots(col = "red", size = 0.1)
```

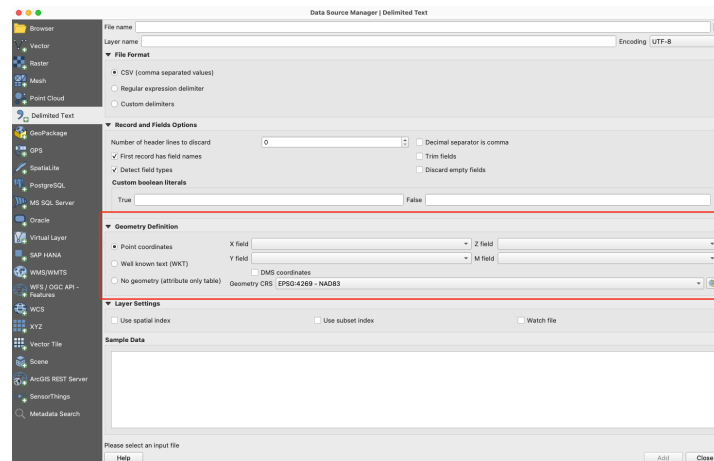
Make Interactive Maps using `leaflet`

```
# Load Required Packages if necessary  
library(dplyr)      # package for r pipes  
library(leaflet)    # package for interactive map  
  
# Define a color palette  
pal <- colorNumeric(  
  palette = c("lightblue", "yellow", "red"), # Improved readable palette  
  domain = your_shapefile$your_variable)  
  
# Build an interactive map  
leaflet(your_shapefile) %>%  
  addProviderTiles("CartoDB.Positron") %>%  
  addPolygons(  
    fillColor = ~pal(your_variable),  
    popup = ~paste(  
      "Census Tract: ", GEOID.x,  
      "<br>Arrests: ", your_variable)) %>%  
  addLegend(  
    pal = pal,  
    values = ~your_variable,  
    title = "Title Here",  
    position = "bottomright"  
  )
```

GIS Research with QGIS

Import Data

- File import:
 - Layer → Add Layer
 - Make sure to use the `...` to get the correct file path and name.
 - Drag and Drop
- .shp/ .csv/
 - .shp file: **Layer → Add Layer → Add Vector Layer**
 - .csv file: **Layer → Add Layer → Add Delimited Text Layer**
 - For **point layers**, use the “Geometry Definition” section and select “Point Coordinates”. The X field is longitude (east/west), and the Y field is latitude (north/south)

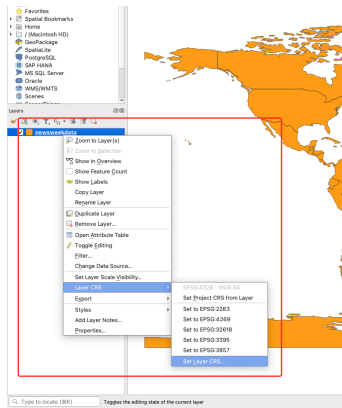


Set CRS

- Set the project CRS
 - right-click layer. Layer CRS → Set Project CRS from Layer
 - Click the icon at the bottom right.



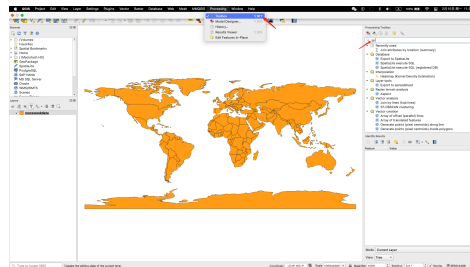
- Set layer CRS: right-click layer. Layer CRS → Set Layer CRS...



Processing with Spatial Layers

For all operations:

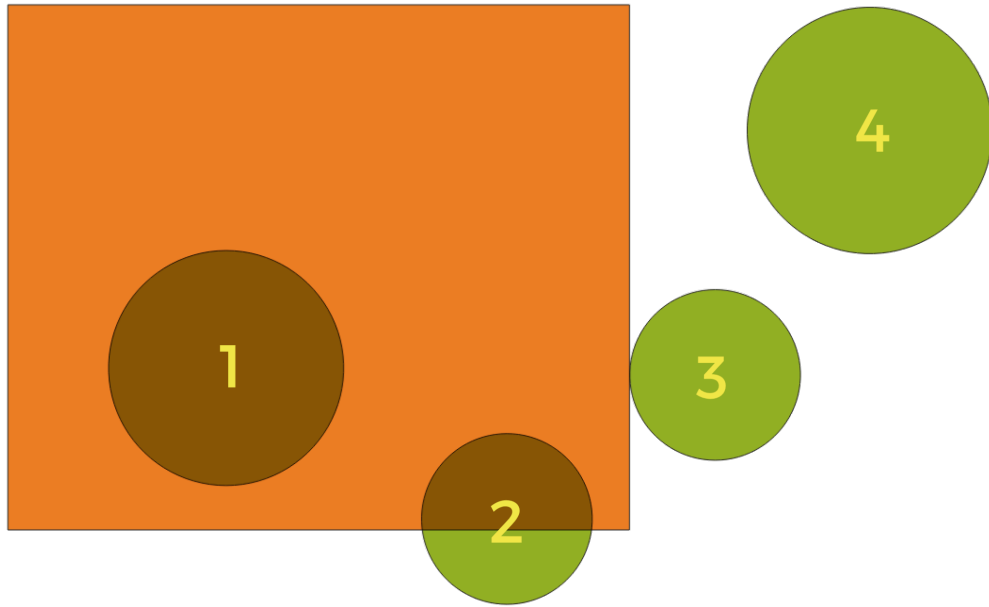
Go to the Navigation Bar at the top of your screen. Click on **Processing > Toolbox**, and perform searches.



Spatial Relationships in QGIS

Detailed document here:

(https://docs.qgis.org/3.40/en/docs/user_manual/processing_algs/qgis/vectorselection.html)



	Description	Result
Intersect	Tests whether a geometry intersects another. Returns 1 (true) if the geometries spatially intersect (share any portion of space - overlap or touch) and 0 if they don't.	Circles 1,2,3
Contain	Returns 1 (true) if and only if no points of b lie in the exterior of a, and at least one point of the interior of b lies in the interior of a. In the picture, no circle is returned, but the rectangle would be if you would look for it the other way around, as it contains circle 1 completely. This is the opposite of are within.	No circles
Disjoint	Returns 1 (true) if the geometries do not share any portion of space (no overlap, not touching). Only circle 4 is returned.	Circle 4
Equal	Returns 1 (true) if and only if geometries are exactly the same. No circles will be returned.	No circles
Touch	Tests whether a geometry touches another. Returns 1 (true) if the geometries have at least one point in common, but their interiors do not intersect. Only circle 3 is returned.	Circle 3
Overlap	Tests whether a geometry overlaps another. Returns 1 (true) if the geometries share space, are of the same dimension, but are not completely contained by each other. Only circle 2 is returned.	Circle 2
Are within	Tests whether a geometry is within another. Returns 1	Circle 1

	(true) if geometry a is completely inside geometry b. Only circle 1 is returned.	
Cross	Returns 1 (true) if the supplied geometries have some, but not all, interior points in common and the actual crossing is of a lower dimension than the highest supplied geometry. For example, a line crossing a polygon will cross as a line (true). Two lines crossing will cross as a point (true). Two polygons cross as a polygon (false). In the picture, no circles will be returned.	No circles

Fix Geometries

- Search for “**fix geometries**” in the processing toolbox

Generate Boundary Lines

- Search for “**dissolve**” in the processing toolbox

Perform Spatial Intersection

- Search for “**Intersection**” in the processing toolbox
- Vector > Geoprocessing Tools > Intersection

Select Spatial Features by Location

- Search for “**Select by location**” in the processing toolbox
- Vector > Research Tools > Select by location

Create Buffers

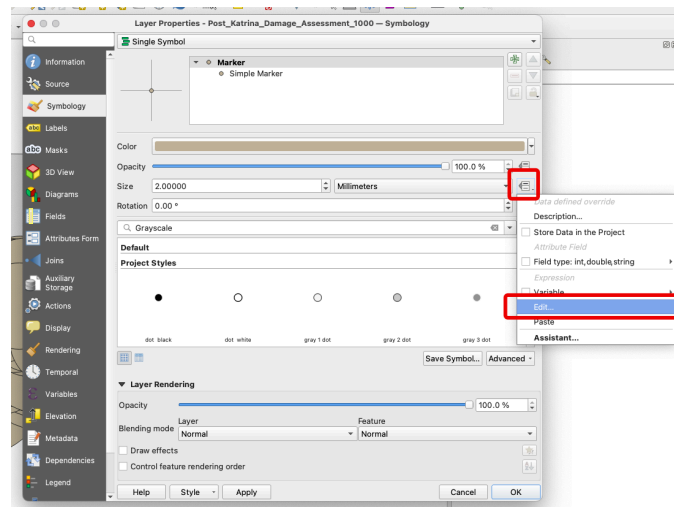
- Search for “**Buffer**” in the processing toolbox
- Vector > Geoprocessing Tools > Buffer

Count Points in Polygon

- Search for “**Count points in polygon**” in the processing toolbox
- Vector > Analysis Tools > Count points in polygon

Set Point Size Based on Variables

- Double-click the point layer to open the Layer Properties window, and go to the Symbology tab.
- Click on the drop-down button next to size, then select “edit”



- In the pop-up window, enter the expression using your variable of interest.
 - You can adjust the size using functions like $/100$, $*10$, $\text{sqrt}()$, or $\ln()$ if the symbols appear too large or too small on the map.

Join Attributes by Location

- Search for “**Join attributes by location**” in the processing toolbox
- Vector > Data Management Tools > Join attributes by location
- There are several types of spatial relationships in QGIS; you can check the usage examples in the previous section: [Spatial Relationships in QGIS](#).

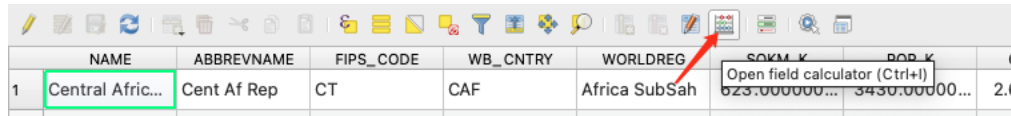
Processing with Tables

SQL in QGIS

- QGIS supports SQL-like expressions for a variety of tasks, including field calculations, feature selection, and creating virtual layers (tables).
- The following sections outline common use cases for processing attribute tables using SQL expressions in QGIS.
- SQL Cheatsheets are widely available online, and AI-generated code can also be helpful.

Add/Change Variables.

Right-click layer. Open Attribute Table. Click the icon at the top.

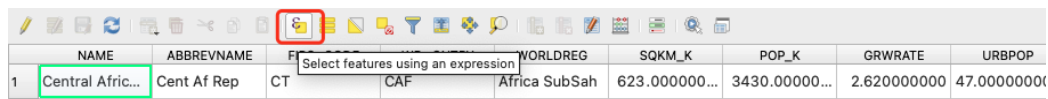


	NAME	ABBREVNNAME	FIPS_CODE	WB_CNTRY	WORLDREG	SQKM_K	POP_K	GRWRATE	URBPOP
1	Central Afric...	Cent Af Rep	CT	CAF	Africa SubSah	623.000000...	3430.00000...	2.6200000000	47.0000000000

- Calculations:
$$("field_name_1" / "field_name_2") * 100.$$
- Calculate area:
\$area
 - The unit of measurement is based on your CRS.
- Change values:
`CASE WHEN "field_name" IS NULL THEN 99 ELSE "field_name" END`
 - Recode missing values into 99
- Recode numerical variables into categorical:
`CASE`
`WHEN "field_name" < 20000 THEN 'group_1'`
`WHEN "field_name" >= 20000 AND "field_name" < 50000 THEN 'group_2'`
`WHEN "field_name" >= 50000 THEN 'group_3'`
`ELSE 'Unknown'`
`END`

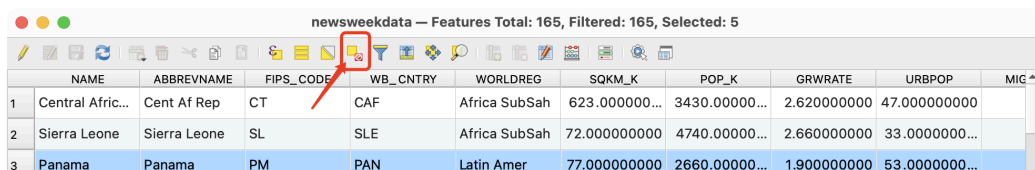
Select Spatial Features Using an Expression

Right-click layer. Open Attribute Table. Click the icon at the top.



	NAME	ABBREVNNAME	FIPS_CODE	WB_CNTRY	WORLDREG	SQKM_K	POP_K	GRWRATE	URBPOP
1	Central Afric...	Cent Af Rep	CT	CAF	Africa SubSah	623.000000...	3430.00000...	2.6200000000	47.0000000000

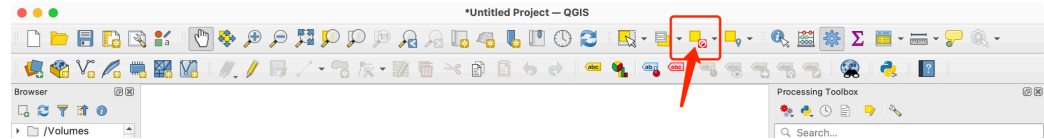
- Select numerical features: `"field_name" = 6`
- Select strings: `"field_name" = 'California'`
- Select multiple categories
 - `"field_name" IN ('California', 'Texas', 'New York')`
 - `"field_name" IN (6, 48, 36)`
- Combining Conditions: `AND/OR/NOT`
- **De-select features**
 - Click on this icon in the attribute table



newsweekdata — Features Total: 165, Filtered: 165, Selected: 5

	NAME	ABBREVNNAME	FIPS_CODE	WB_CNTRY	WORLDREG	SQKM_K	POP_K	GRWRATE	URBPOP	MIG
1	Central Afric...	Cent Af Rep	CT	CAF	Africa SubSah	623.000000...	3430.00000...	2.6200000000	47.0000000000	
2	Sierra Leone	Sierra Leone	SL	SLE	Africa SubSah	72.0000000000	4740.00000...	2.6600000000	33.00000000...	
3	Panama	Panama	PM	PAN	Latin Amer	77.0000000000	2660.00000...	1.9000000000	53.00000000...	

- OR Click this icon in the main UI



Join Tables by ID

- Open: Double-click layer. OR right-click layer, go to “Properties...” . Then go to “Joins”

Data Resources

ACS Data

- <https://www.socialexplorer.com/explore-tables>
- Data dictionary / Codebook:
https://www.socialexplorer.com/data/ACS2022_5yr/metadata/?ds=SE

US Shapefiles

- <https://www.census.gov/cgi-bin/geo/shapefiles/index.php>

Appendix 1: State FIPS Code

FIPS Code	State Name	FIPS Code	State Name
1	ALABAMA	31	NEBRASKA
2	ALASKA	32	NEVADA
4	ARIZONA	33	NEW HAMPSHIRE
5	ARKANSAS	34	NEW JERSEY
6	CALIFORNIA	35	NEW MEXICO
8	COLORADO	36	NEW YORK
9	CONNECTICUT	37	NORTH CAROLINA
10	DELAWARE	38	NORTH DAKOTA
11	DISTRICT OF COLUMBIA	39	OHIO
12	FLORIDA	40	OKLAHOMA
13	GEORGIA	41	OREGON
15	HAWAII	42	PENNSYLVANIA
16	IDAHO	44	RHODE ISLAND
17	ILLINOIS	45	SOUTH CAROLINA
18	INDIANA	46	SOUTH DAKOTA
19	IOWA	47	TENNESSEE
20	KANSAS	48	TEXAS
21	KENTUCKY	49	UTAH
22	LOUISIANA	50	VERMONT
23	MAINE	51	VIRGINIA
24	MARYLAND	53	WASHINGTON
25	MASSACHUSETTS	54	WEST VIRGINIA
26	MICHIGAN	55	WISCONSIN
27	MINNESOTA	56	WYOMING
28	MISSISSIPPI		
29	MISSOURI		
30	MONTANA		

Appendix 2: SQL Cheatsheet

SQL CHEAT SHEET <http://www.sqltutorial.org>



QUERYING DATA FROM A TABLE

SELECT c1, c2 FROM t;
Query data in columns c1, c2 from a table

SELECT * FROM t;
Query all rows and columns from a table

**SELECT c1, c2 FROM t
WHERE condition;**
Query data and filter rows with a condition

**SELECT DISTINCT c1 FROM t
WHERE condition;**
Query distinct rows from a table

**SELECT c1, c2 FROM t
ORDER BY c1 ASC [DESC];**
Sort the result set in ascending or descending order

**SELECT c1, c2 FROM t
ORDER BY c1
LIMIT n OFFSET offset;**
Skip *offset* of rows and return the next n rows

**SELECT c1, aggregate(c2)
FROM t
GROUP BY c1;**
Group rows using an aggregate function

**SELECT c1, aggregate(c2)
FROM t
GROUP BY c1
HAVING condition;**
Filter groups using HAVING clause

QUERYING FROM MULTIPLE TABLES

**SELECT c1, c2
FROM t1
INNER JOIN t2 ON condition;**
Inner join t1 and t2

**SELECT c1, c2
FROM t1
LEFT JOIN t2 ON condition;**
Left join t1 and t2

**SELECT c1, c2
FROM t1
RIGHT JOIN t2 ON condition;**
Right join t1 and t2

**SELECT c1, c2
FROM t1
FULL OUTER JOIN t2 ON condition;**
Perform full outer join

**SELECT c1, c2
FROM t1
CROSS JOIN t2;**
Produce a Cartesian product of rows in tables

**SELECT c1, c2
FROM t1, t2;**
Another way to perform cross join

**SELECT c1, c2
FROM t1 A
INNER JOIN t2 B ON condition;**
Join t1 to itself using INNER JOIN clause

USING SQL OPERATORS

**SELECT c1, c2 FROM t1
UNION [ALL]
SELECT c1, c2 FROM t2;**
Combine rows from two queries

**SELECT c1, c2 FROM t1
INTERSECT
SELECT c1, c2 FROM t2;**
Return the intersection of two queries

**SELECT c1, c2 FROM t1
MINUS
SELECT c1, c2 FROM t2;**
Subtract a result set from another result set

**SELECT c1, c2 FROM t1
WHERE c1 [NOT] LIKE pattern;**
Query rows using pattern matching %, _

**SELECT c1, c2 FROM t
WHERE c1 [NOT] IN value_list;**
Query rows in a list

**SELECT c1, c2 FROM t
WHERE c1 BETWEEN low AND high;**
Query rows between two values

**SELECT c1, c2 FROM t
WHERE c1 IS [NOT] NULL;**
Check if values in a table is NULL or not